and exceptions so that the code clearly articulates the rules that are being enforced by the smart contract for implementing trust and integrity. The special keyword `modifier` can be used by a smart contract auditor (manual or automatic) to make sure that all the rules are defined up front and used as expected. When a modifier representing a rule is defined, it can be used any number of times, like a function call. This pattern allows you to review the code locations where rules are applied easily.

Now let's find out how a modifier is invoked within the code. Listing 3.4 shows an actual function, `register()`, using the modifier `validPhase`. The modifier is located in the header of the function. The traditional code for checking the condition is also shown in the listing's second line, commented out. You can see the elegance of the modifier as opposed to this line (the `if` statement). A review of the function header demonstrates that the state of the voting process is checked (it must be `Phase.Regs`) before anything is done in the function.

Listing 3.4 Use of a modifier

```
function register(address voter) public validPhase(Phase.Regs) {          ❶
        //if (state != Phase.Regs) {revert();}                            ❷
        if (msg.sender != chairperson || voters[toVoter].voted) return;
        voters[voter].weight = 1;
        voters[voter].voted = false;
        ...
    }
```

❶ Modifier in the header of function; if the condition is not met, revert the transaction

❷ Equivalent traditional code

The use of modifiers as trust implementers (intermediaries) results in design principle 6.

> **DESIGN PRINCIPLE 6** Implement the verification and validation needed for trust intermediation by using modifiers specifying the rules and conditions in